Abstract Data Types

Comp Sci 214, Spring 2025

Don’t forget to check in on Youhere!

Reminders

- Homework 1 due today
  - Can spend one of your late tokens to have until Saturday
  - Feedback will come on Sunday
  - Then fix what you got wrong, and resubmit by next Thu!
  - Self-evaluation will go out on Monday

- Homework 2 out now
  - Covering today's material

Abstract Data Types (ADTs)

- Due to Barbara Liskov in 1974
  - She received the 2008 Turing Award for this work (and more)
- One of the most important advances in programming
  - On par with structured programming and automatic memory management
- Will be one of our guiding notions in this class

What is an ADT?

An ADT defines:

- A set of (abstract) values
- A set of (abstract) operations on those values

An ADT omits:

- How the values are concretely represented
- How the operations work

Omissions = Axes of Freedom!

- Can choose between many different representations!
- And/or different implementations for operations!
- Each with their own tradeoffs

ADT: Stack

Meaning: A stack of plates, “undo” actions in an editor, or a program’s call stack

Abstract values look like: $\boxed{|3, 4, 5\rangle}$

ADT: Stack

Meaning: A stack of plates, “undo” actions in an editor, or a program’s call stack

Abstract values look like: $\boxed{|3, 4, 5\rangle}$

Abstract operations signature:

- push(Stack, Element): None
- pop(Stack): Element
- empty?(Stack): Bool

ADT: Stack

Meaning: A stack of plates, “undo” actions in an editor, or a program’s call stack

Abstract values look like: $\boxed{|3, 4, 5\rangle}$

Abstract operations signature, as a DSSL2 interface:

```
interface STACK:
    def push(self, element)
    def pop(self)
    def empty?(self)
```

- DSSL2 interface ≈ C++ abstract class
- Abstractly *specifies* operations, but not concrete implementations
- Classes can *implement* interfaces to fill those in

ADT: Stack

Meaning: A stack of plates, “undo” actions in an editor, or a program’s call stack

Abstract values look like: $\boxed{|3, 4, 5\rangle}$

Abstract operations signature, with contracts:

```
interface STACK[T]: # E.g., stack of ints, of strings, ...
    def push(self, element: T) -> NoneC
    def pop(self) -> T
    def empty?(self) -> bool?
```

- Contracts are a way to check type-like constraints during program execution
- We may use some in examples or in starter code, but you don’t need to write new ones (unless you want to)
- See supplementary video on Canvas (and docs)

ADT: Stack, Example

$\boxed{|3, 4, 5\rangle}$.push(6)

ADT: Stack, Example

$\boxed{|3, 4, 5\rangle}$.push(6) $\Rightarrow$ None

Stack becomes: $\boxed{|3, 4, 5, 6\rangle}$

ADT: Stack, Example

$\boxed{|3, 4, 5\rangle}$.push(6) $\Rightarrow$ None

Stack becomes: $\boxed{|3, 4, 5, 6\rangle}$

$\boxed{|3, 4, 5, 6\rangle}$.pop() $\Rightarrow$???

ADT: Stack, Example

$\boxed{|3,4,5\rangle}$.push(6) $\Rightarrow$ None

Stack becomes: $\boxed{|3,4,5,6\rangle}$

All the action happens at the pointy end (top)

$\boxed{|3,4,5,6\rangle}$.pop() $\Rightarrow$ 6

Stack becomes: $\boxed{|3,4,5\rangle}$

Last element to come *in* is the *first* to come *out*! Last-in, first-out. (**LIFO**)

The ADT definition is enough for us to know *what* it does!

That's enough to be able to use it.

But we don't know *how* it does it!

Omissions = axes of freedom! Many possible solutions.

ADT: Queue

Meaning: A line to check out at the grocery store, a playlist in a music player, or number tickets while waiting at the DMV

Abstract values look like: $\boxed{\langle 3, 4, 5 \langle}$

Abstract operations signature, with contracts:

```
interface QUEUE[T]:
    def enqueue(self, element: T) -> NoneC
    def dequeue(self) -> T
    def empty?(self) -> bool?
```

ADT: Queue, Example

$\boxed{\langle 3, 4, 5 \langle}$.enqueue(6)

ADT: Queue, Example

$\boxed{\langle 3, 4, 5 \langle}$.enqueue(6) $\Rightarrow$ `None`

Queue becomes: $\boxed{\langle 3, 4, 5, 6 \langle}$

ADT: Queue, Example

$\boxed{\langle 3, 4, 5 \langle}$.enqueue(6) $\Rightarrow$ `None`

Queue becomes: $\boxed{\langle 3, 4, 5, 6 \langle}$

$\boxed{\langle 3, 4, 5, 6 \langle}$.dequeue() $\Rightarrow$???

ADT: Queue, Example

$\boxed{\langle 3, 4, 5 \langle}$.enqueue(6) $\Rightarrow$ None

Queue becomes: $\boxed{\langle 3, 4, 5, 6 \langle}$

$\boxed{\langle 3, 4, 5, 6 \langle}$.dequeue() $\Rightarrow$ 3

Queue becomes: $\boxed{\langle 4, 5, 6 \langle}$

Action happens on both sides (front and back)

First element to come *in* is the *first* to come *out*!
First-in, first-out. (**FIFO**)

Stack versus Queue

```
interface STACK[T]:
    def push(self, element: T) -> NoneC
    def pop(self) -> T
    def empty?(self) -> bool?

interface QUEUE[T]:
    def enqueue(self, element: T) -> NoneC
    def dequeue(self) -> T
    def empty?(self) -> bool?
```

Same thing except for the names!

So what's the difference?

Clearly we're missing something…

Adding laws

In addition to abstract values and abstract operations, ADTs will define *laws* for operations that specify correct behavior:

- Inputs
- Outputs
- Changes to abstract values

So *implementers* know what their data structures should do

So *users* know what to expect when they use them

Laws are an agreement between these two parties

Adding laws

$$\{p\}\ f(x) \Rightarrow y\ \{q\}$$

means that if precondition p is true
then when we run some code $f(x)$
it will produce some value y as its result
and postcondition q will be true afterward.

Adding laws

$$\{p\}\ f(x) \Rightarrow y\ \{q\}$$

means that if precondition p is true
then when we run some code $f(x)$
it will produce some value y as its result
and postcondition q will be true afterward.

Examples:

$$\{a = [2, 4, 6, 8]\}\ a[2] \Rightarrow 6\ \{a = [2, 4, 6, 8]\}$$

$$\{a = [2, 4, 6, 8]\}\ a[2] = 19 \Rightarrow \mathsf{None}\ \{a = [2, 4, 19, 8]\}$$

Adding laws

$$\{p\}\ f(x) \Rightarrow y\ \{q\}$$

means that if precondition p is true
then when we run some code $f(x)$
it will produce some value y as its result
and postcondition q will be true afterward.

Examples:

$$\{a = [2, 4, 6, 8]\}\ a[2] \Rightarrow 6\ \{a = [2, 4, 6, 8]\}$$

$$\{a = [2, 4, 6, 8]\}\ a[2] = 19 \Rightarrow \texttt{None}\ \{a = [2, 4, 19, 8]\}$$

This notation is called Hoare triples, after C. A. R. (Tony) Hoare

- 1980 Turing award, quicksort, concurrency, etc.

Note: this is not code, it's *math* that says what code should do.

ADT: Stack (LIFO)

Abstract values look like: $\boxed{|3,4,5\rangle}$

Abstract operations signature:

```
interface STACK[T]:
    def push(self, element: T) -> NoneC
    def pop(self) -> T
    def empty?(self) -> bool?
```

Laws:

$$\left\{s = \boxed{|\rangle}\right\}\ s.empty?() \Rightarrow \texttt{True}\ \{\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k, e_{k+1}\rangle}\right\}\ s.empty?() \Rightarrow \texttt{False}\ \{\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k\rangle}\right\}\ s.push(e) \Rightarrow \texttt{None}\ \left\{s = \boxed{|e_1, \ldots, e_k, e\rangle}\right\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k, e_{k+1}\rangle}\right\}\ s.pop() \Rightarrow e_{k+1}\ \left\{s = \boxed{|e_1, \ldots, e_k\rangle}\right\}$$

ADT: Stack (LIFO)

Abstract values look like: $\boxed{|3, 4, 5\rangle}$

Abstract operations signature:

```
interface STACK[T]:
    def push(self, element: T) -> NoneC
    def pop(self) -> T
    def empty?(self) -> bool?
```

Laws:

$$\left\{s = \boxed{|\rangle}\right\}\ s.empty?() \Rightarrow \texttt{True}\ \{\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k, e_{k+1}\rangle}\right\}\ s.empty?() \Rightarrow \texttt{False}\ \{\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k\rangle}\right\}\ s.push(e) \Rightarrow \texttt{None}\ \left\{s = \boxed{|e_1, \ldots, e_k, e\rangle}\right\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k, e_{k+1}\rangle}\right\}\ s.pop() \Rightarrow e_{k+1}\ \left\{s = \boxed{|e_1, \ldots, e_k\rangle}\right\}$$

Anything missing?

ADT: Stack (LIFO)

Abstract values look like: $\boxed{|3, 4, 5\rangle}$

Abstract operations signature:

```
interface STACK[T]:
    def push(self, element: T) -> NoneC
    def pop(self) -> T
    def empty?(self) -> bool?
```

Laws:

$$\left\{s = \boxed{|\rangle}\right\}\ s.empty?() \Rightarrow \texttt{True}\ \{\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k, e_{k+1}\rangle}\right\}\ s.empty?() \Rightarrow \texttt{False}\ \{\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k\rangle}\right\}\ s.push(e) \Rightarrow \texttt{None}\ \left\{s = \boxed{|e_1, \ldots, e_k, e\rangle}\right\}$$

$$\left\{s = \boxed{|e_1, \ldots, e_k, e_{k+1}\rangle}\right\}\ s.pop() \Rightarrow e_{k+1}\ \left\{s = \boxed{|e_1, \ldots, e_k\rangle}\right\}$$

Laws don't mention? $\rightarrow$ Anything could happen! Can't assume.

ADT: Queue (FIFO)

Abstract values look like: $\boxed{\langle 3, 4, 5 \langle}$

Abstract operations signature:

```
interface QUEUE[T]:
    def enqueue(self, element: T) -> NoneC
    def dequeue(self) -> T
    def empty?(self) -> bool?
```

Laws:

$$\left\{q = \boxed{\langle\langle}\right\}\ q.empty?() \Rightarrow \texttt{True}\ \{\}$$

$$\left\{q = \boxed{\langle e_1, \ldots, e_k, e_{k+1} \langle}\right\}\ q.empty?() \Rightarrow \texttt{False}\ \{\}$$

$$\left\{q = \boxed{\langle e_1, \ldots, e_k \langle}\right\}\ q.enqueue(e) \Rightarrow \texttt{None}\ \left\{q = \boxed{\langle e_1, \ldots, e_k, e \langle}\right\}$$

$$\left\{q = \boxed{\langle e_1, e_2, \ldots, e_k \langle}\right\}\ q.dequeue() \Rightarrow e_1\ \left\{q = \boxed{\langle e_2, \ldots, e_k \langle}\right\}$$

Let's take stock

At this point we have seen, for both stacks and queues:

- Abstract values
- Abstract operations
- Laws

That's all we need to be able to *use* stacks and queues

- I.e., if someone handed us a piece of code and said "Here's a stack", we'd be good to go.

But that's not enough to be able to *build* stacks and queues

- What's our representation?
- How do we write our operations?

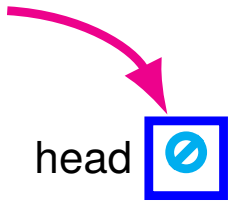
We'll look at that next.

Stop! Question time!

Before we move on to implementations...

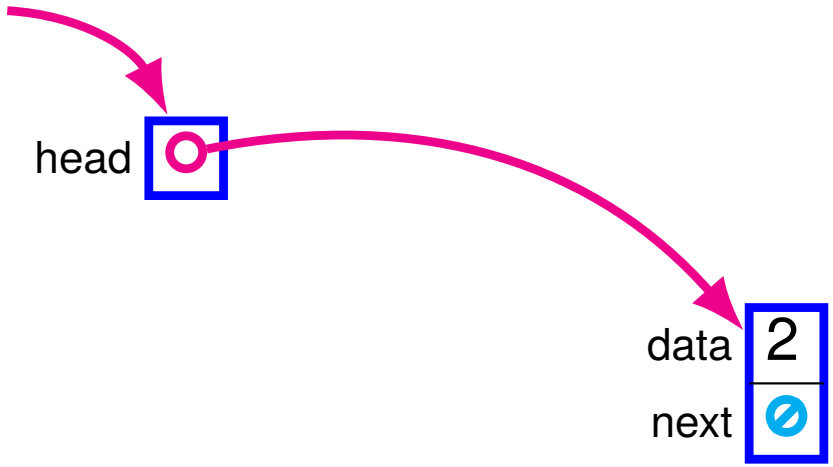
Anything unclear so far?
Anything I missed?

Stack implementation: linked list



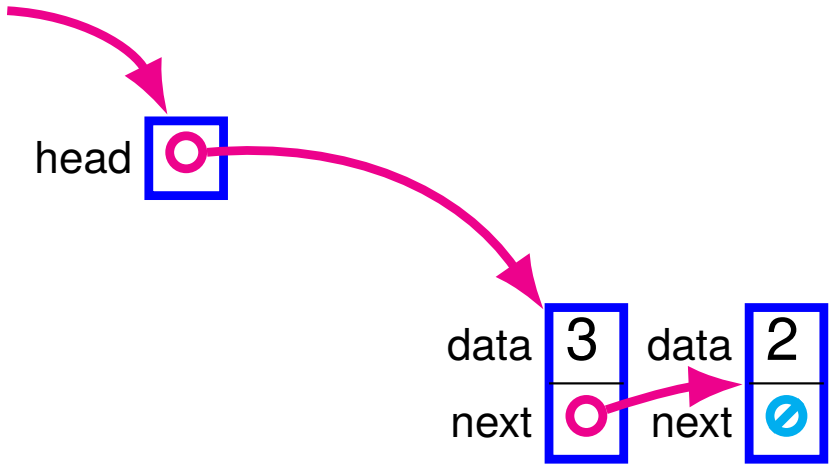
let s = ListStack()

Stack implementation: linked list




```
let s = ListStack()
s.push(2)
```

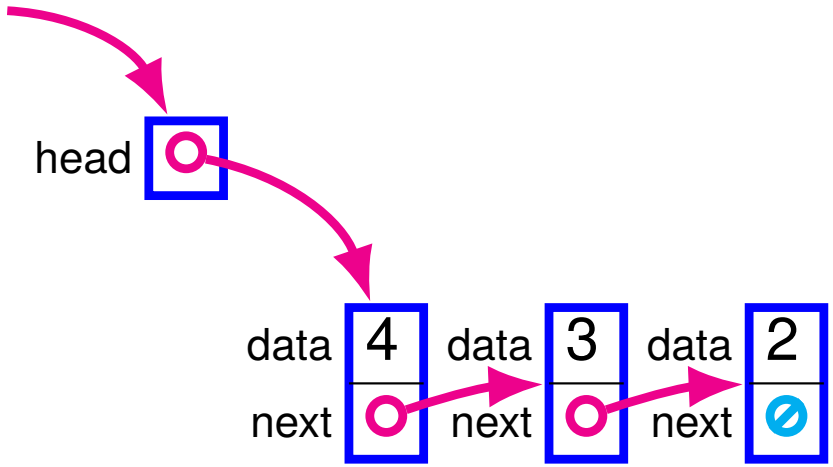
Stack implementation: linked list




```
let s = ListStack()
s.push(2)
s.push(3)
```

push = add to start of list

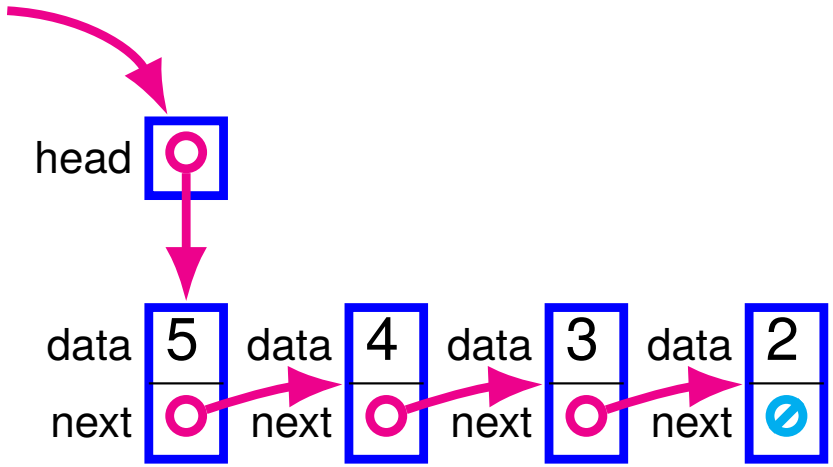
Stack implementation: linked list




```
let s = ListStack()
s.push(2)
s.push(3)
s.push(4)
```

push = add to start of list

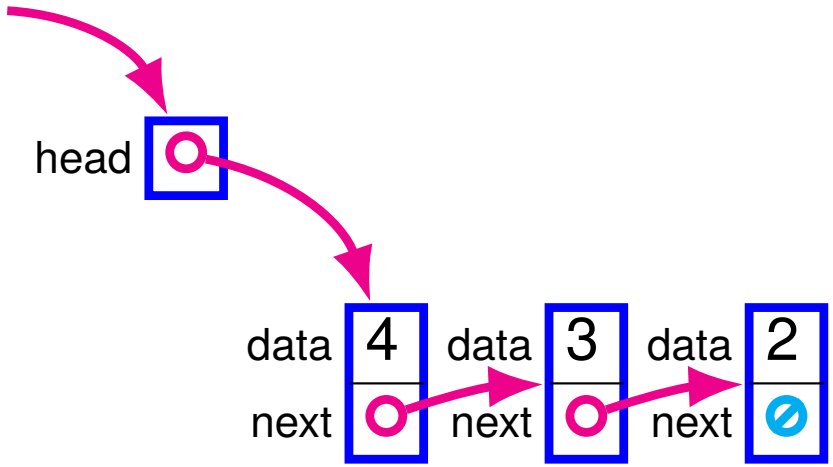
Stack implementation: linked list




```
let s = ListStack()
s.push(2)
s.push(3)
s.push(4)
s.push(5)
```

push = add to start of list

Stack implementation: linked list

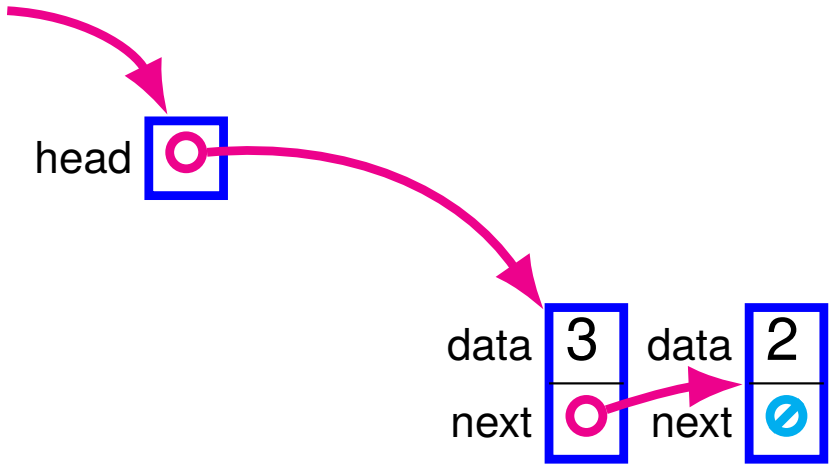



```
let s = ListStack()
s.push(2)
s.push(3)
s.push(4)
s.push(5)
s.pop()
```

push = add to start of list

pop = remove from front of list

Stack implementation: linked list

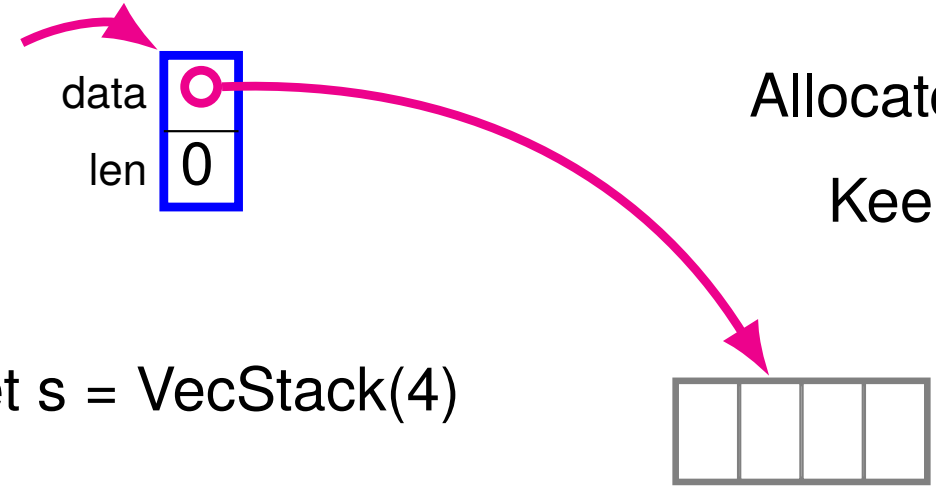



```
let s = ListStack()
s.push(2)
s.push(3)
s.push(4)
s.push(5)
s.pop()
s.pop()
```

push = add to start of list

pop = remove from front of list

Stack implementation: array

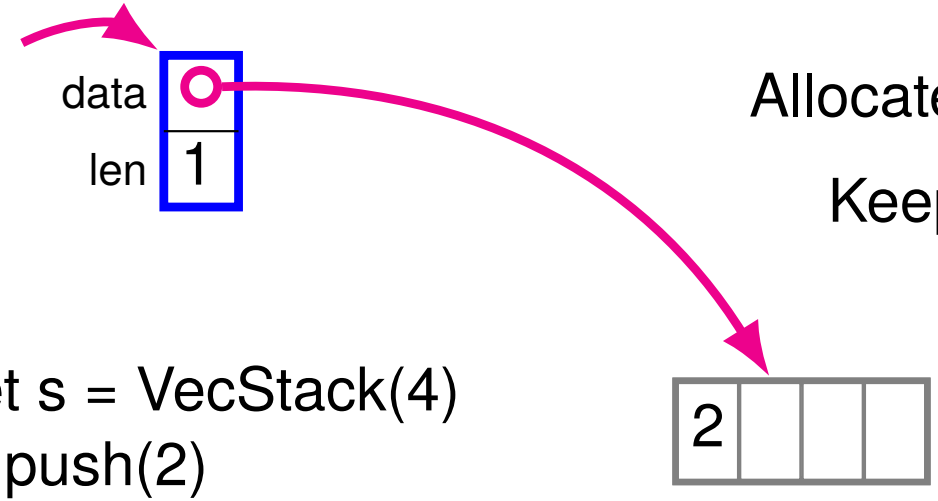


Allocate array with spare capacity.

Keep track of # of elements.

let s = VecStack(4)

Stack implementation: array

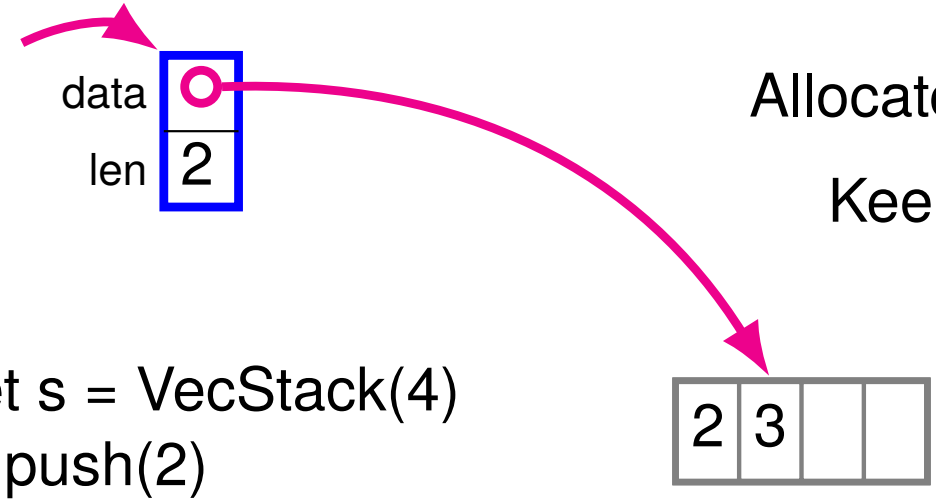


Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
```

Stack implementation: array

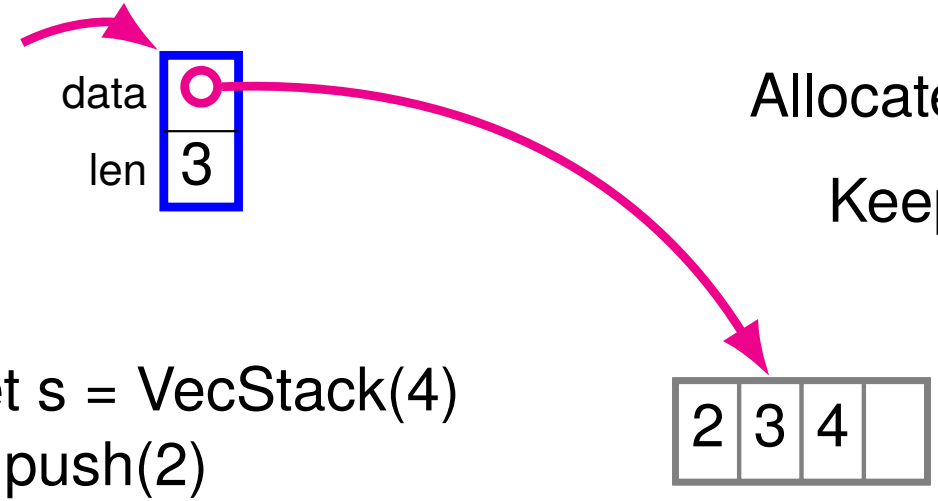


Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
s.push(3)
```

Stack implementation: array

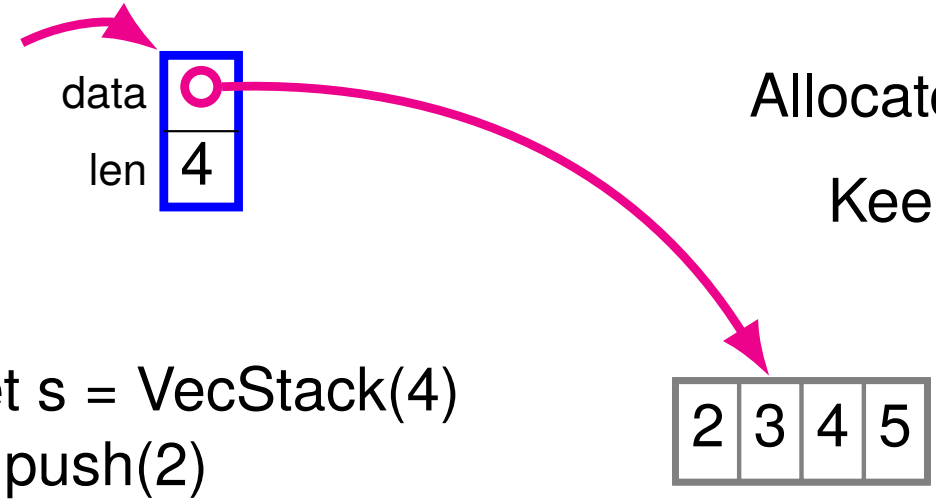


Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
s.push(3)
s.push(4)
```

Stack implementation: array

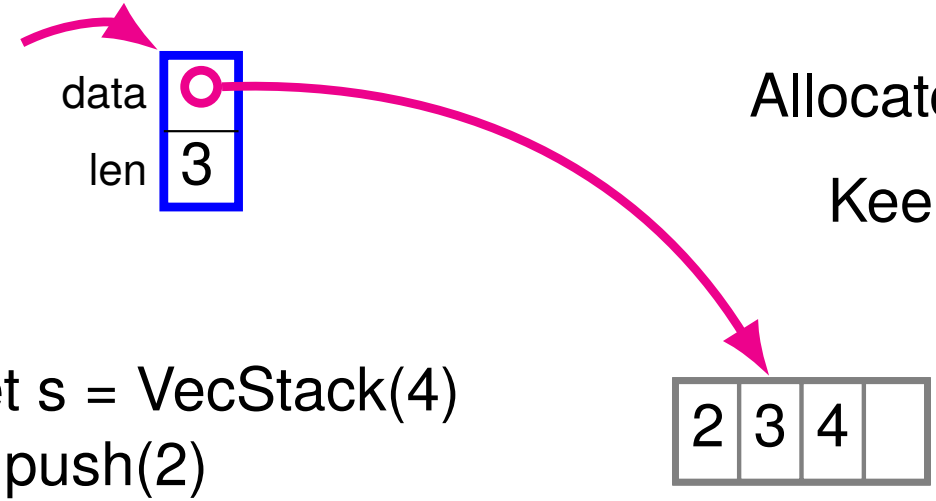


Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
s.push(3)
s.push(4)
s.push(5)
```

Stack implementation: array

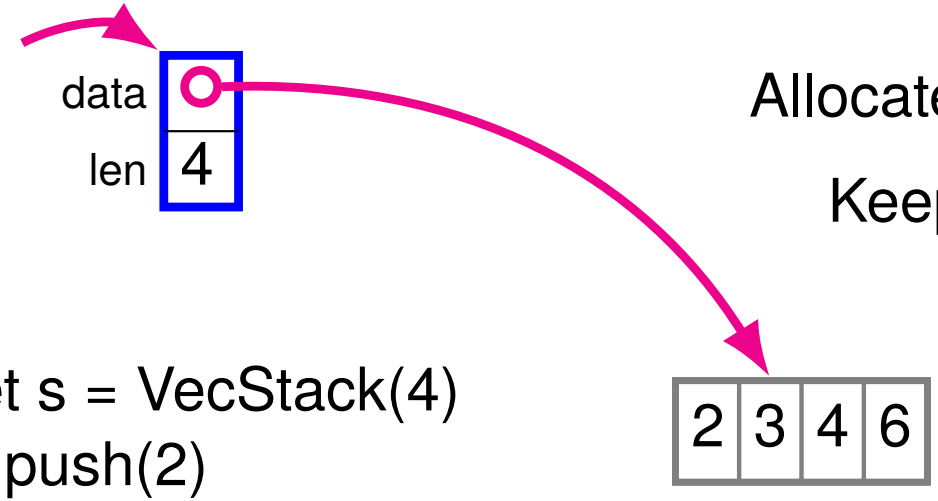


Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
s.push(3)
s.push(4)
s.push(5)
s.pop()
```

Stack implementation: array

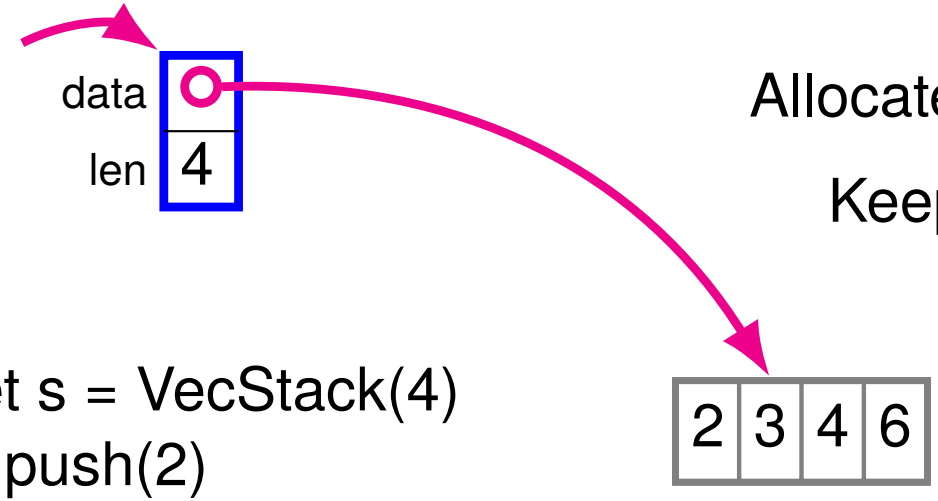


Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
s.push(3)
s.push(4)
s.push(5)
s.pop()
s.push(6)
```

Stack implementation: array



Allocate array with spare capacity.

Keep track of # of elements.

```
let s = VecStack(4)
s.push(2)
s.push(3)
s.push(4)
s.push(5)
s.pop()
s.push(6)
s.push(7)
```

With an array, capacity is an issue.

→ Dyn. arrays (last time) can fix (with some caveats; cf. lec 17).

Tradeoffs: linked list stack versus array stack

Time comparison

- All operations are always cheap for both versions
  - Cheap = number of elements doesn't affect cost
  - Expensive = cost increases with number of elements

- Linked list version: add and remove at start
- Array version: read and write array elements
  - Well, except if array fills up
  - Then either expensive or impossible

- We'll make costs accounting precise in the next lecture

Space comparison

- Linked list: can always add new elts (until memory full)
- Array: has a fixed size (or must reallocate)

ADT: Queue (FIFO)

Abstract values look like: $\boxed{\langle 3, 4, 5 \langle}$

Abstract operations signature:

```
interface QUEUE[T]:
    def enqueue(self, element: T) -> NoneC
    def dequeue(self) -> T
    def empty?(self) -> bool?
```

Laws:

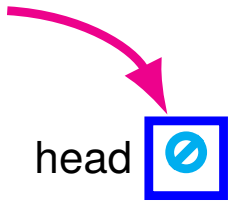
$$\left\{q = \boxed{\langle\langle}\right\}\ q.empty?() \Rightarrow \texttt{True}\ \{\}$$

$$\left\{q = \boxed{\langle e_1, \ldots, e_k, e_{k+1} \langle}\right\}\ q.empty?() \Rightarrow \texttt{False}\ \{\}$$

$$\left\{q = \boxed{\langle e_1, \ldots, e_k \langle}\right\}\ q.enqueue(e) \Rightarrow \texttt{None}\ \left\{q = \boxed{\langle e_1, \ldots, e_k, e \langle}\right\}$$

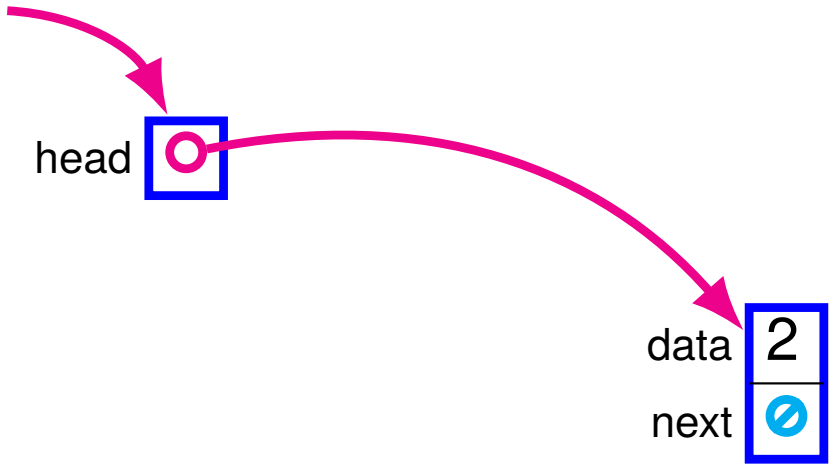
$$\left\{q = \boxed{\langle e_1, e_2, \ldots, e_k \langle}\right\}\ q.dequeue() \Rightarrow e_1\ \left\{q = \boxed{\langle e_2, \ldots, e_k \langle}\right\}$$

Queue implementation: linked list?



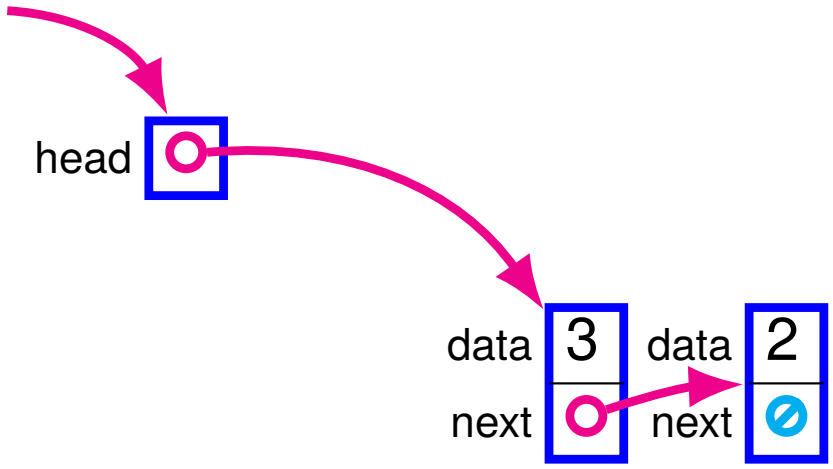
let q = ListQueue()

Queue implementation: linked list?




```
let q = ListQueue()
q.enqueue(2)
```

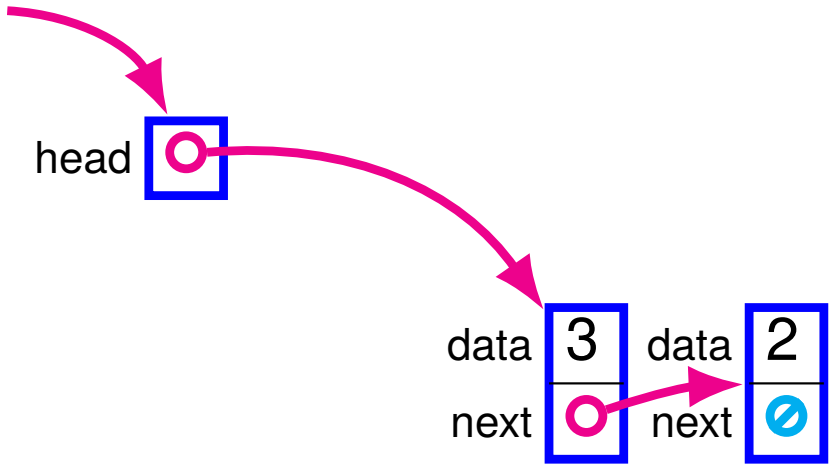
Queue implementation: linked list?



let q = ListQueue()
q.enqueue(2)
q.enqueue(3)

Add at the head. **QUIZ**: cheap?

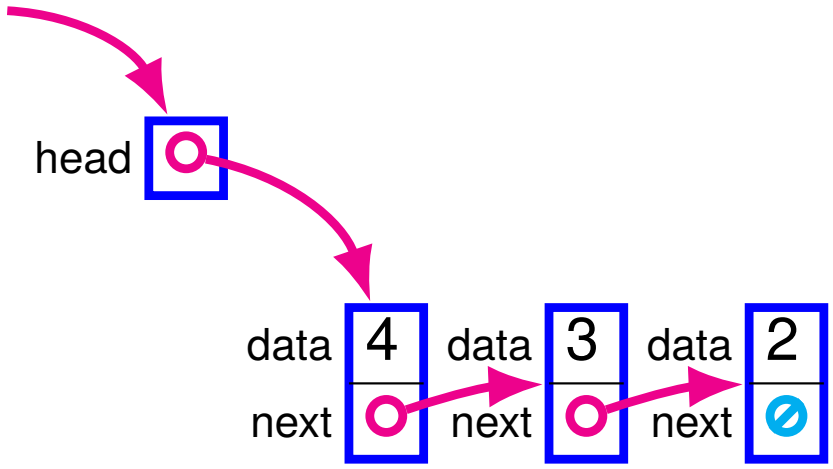
Queue implementation: linked list?



let q = ListQueue()
q.enqueue(2)
q.enqueue(3)

Add at the head. Cheap.

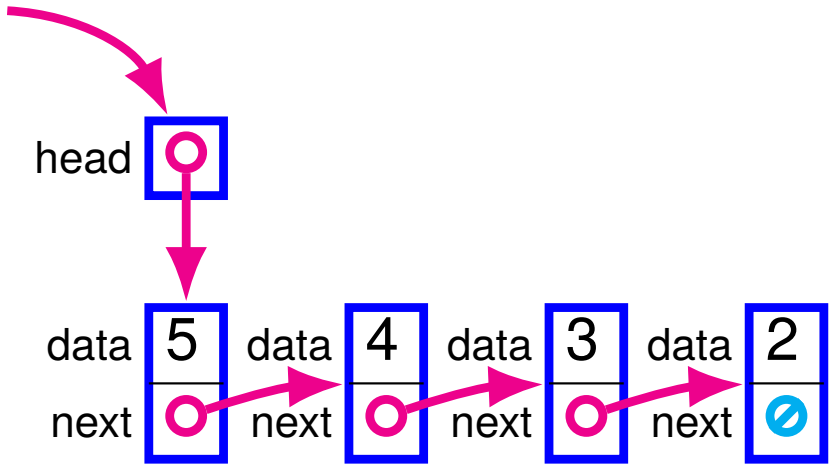
Queue implementation: linked list?




```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
```

Add at the head. Cheap.

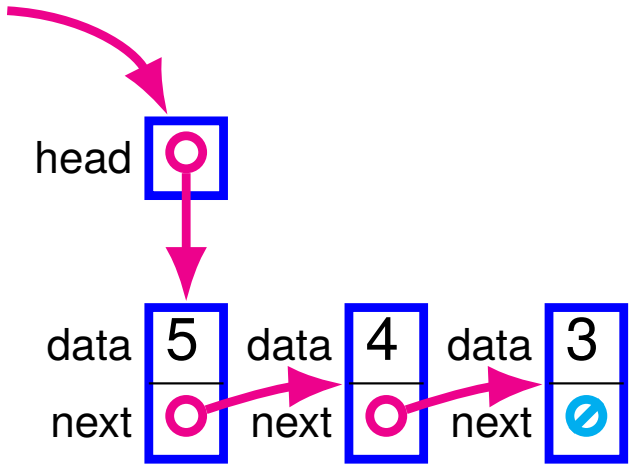
Queue implementation: linked list?




```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
```

Add at the head. Cheap.

Queue implementation: linked list?

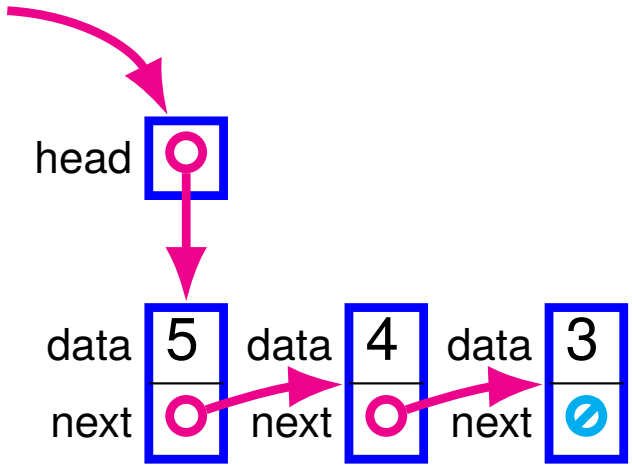



```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at the head. Cheap.

Remove at the tail. **QUIZ**: cheap?

Queue implementation: linked list?

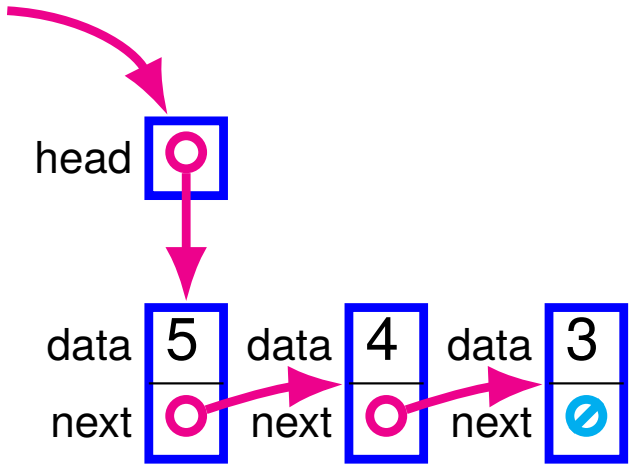



```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at the head. Cheap.

Remove at the tail. Expensive!

Queue implementation: linked list?



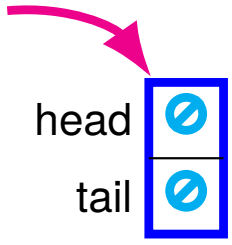

```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at the head. Cheap.

Remove at the tail. Expensive!

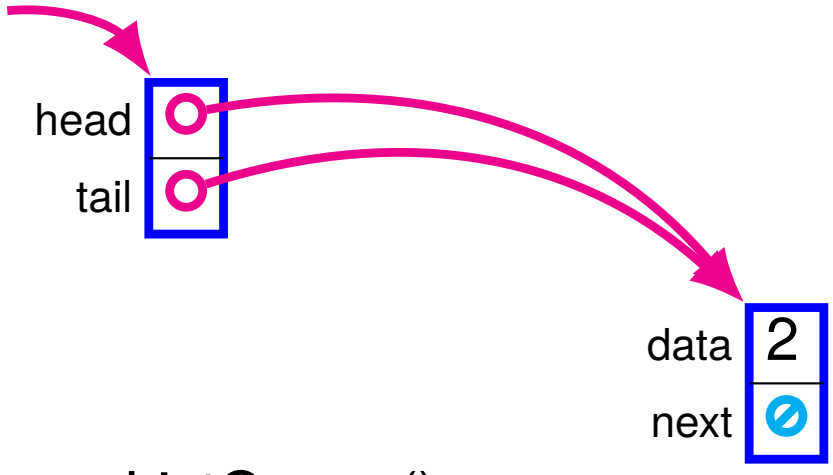
How could we fix this?

Queue implementation: SLL with tail pointer



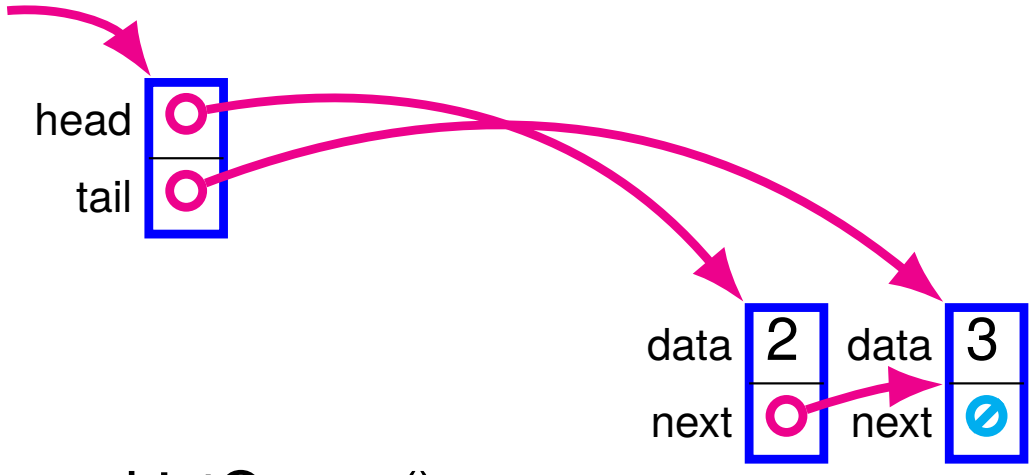
let q = ListQueue()

Queue implementation: SLL with tail pointer



let q = ListQueue()
q.enqueue(2)

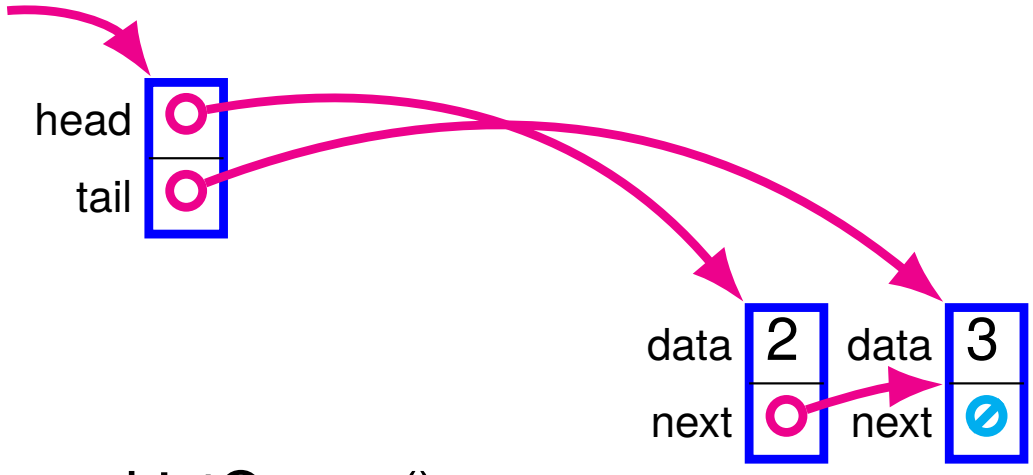
Queue implementation: SLL with tail pointer



let q = ListQueue()
q.enqueue(2)
q.enqueue(3)

Add at the tail. **QUIZ**: cheap?

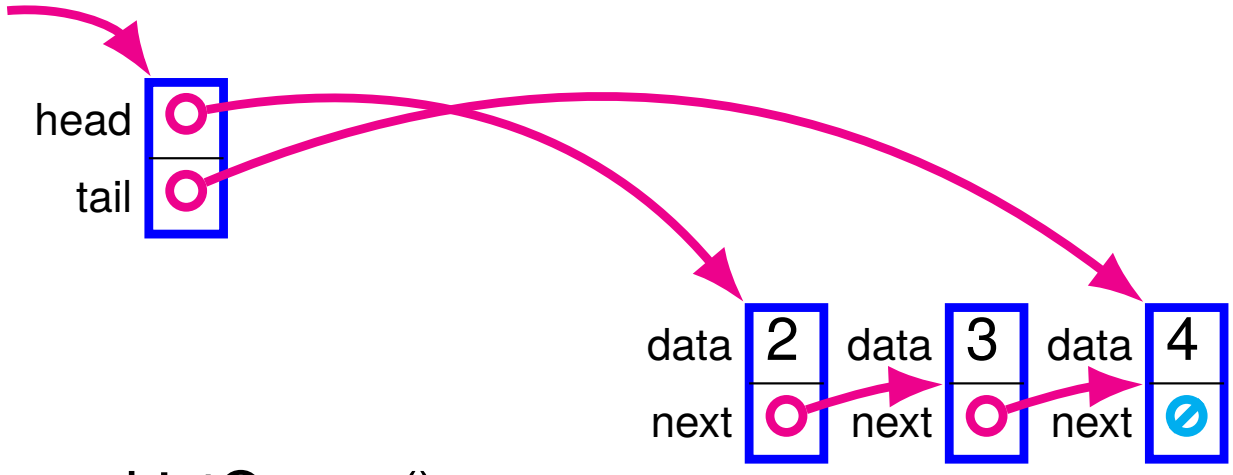
Queue implementation: SLL with tail pointer




```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
```

Add at the tail. Cheap!

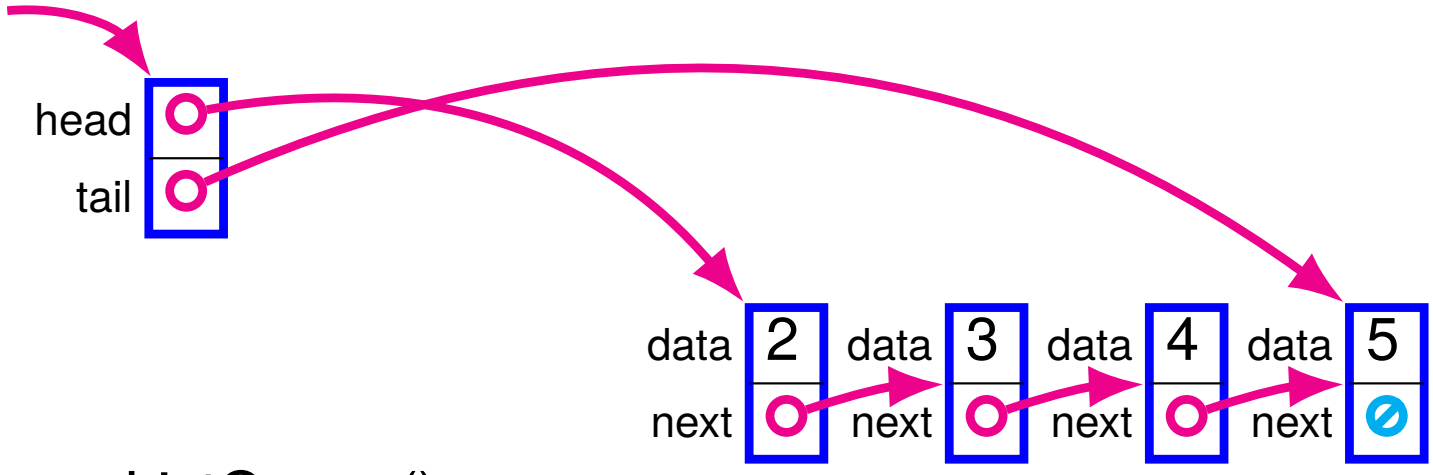
Queue implementation: SLL with tail pointer




```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
```

Add at the tail. Cheap!

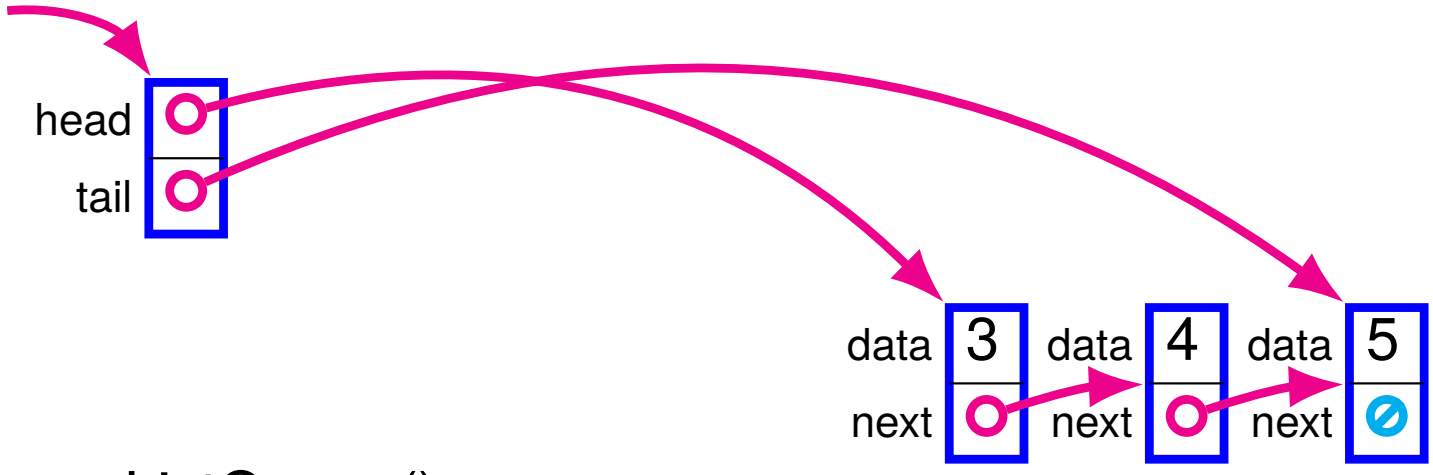
Queue implementation: SLL with tail pointer




```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
```

Add at the tail. Cheap!

Queue implementation: SLL with tail pointer

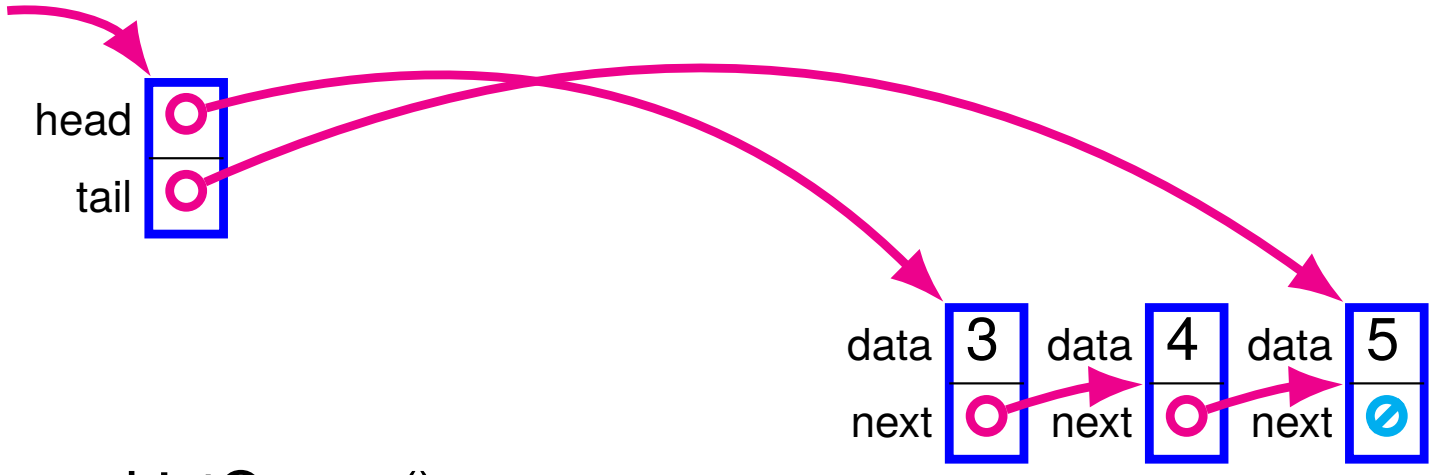



```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at the tail. Cheap!

Remove at the head! **QUIZ**: cheap?

Queue implementation: SLL with tail pointer




```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at the tail. Cheap!

Remove at the head! Also cheap!

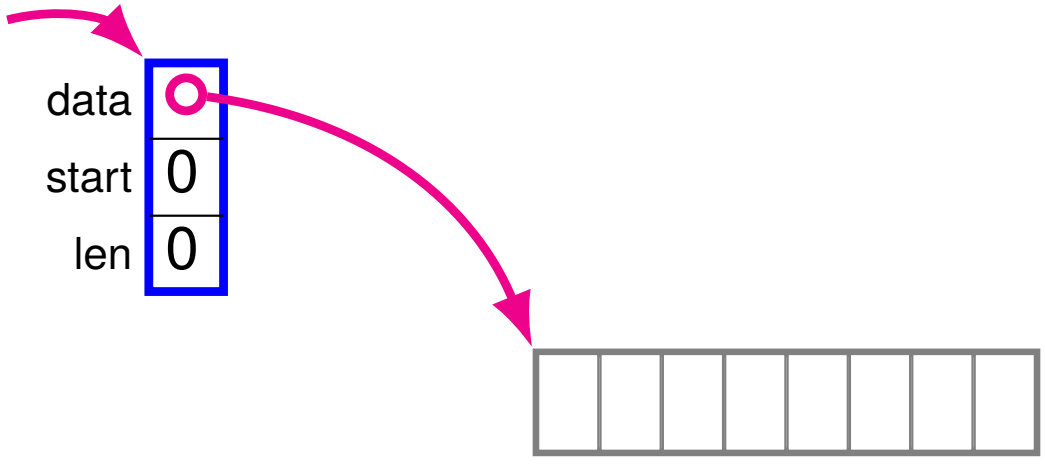
Queue implementation: SLL with tail pointer


```
let q = ListQueue()
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
q.dequeue()
```

Add at the tail. Cheap!

Remove at the head! Also cheap!

Queue implementation: array (*ring buffer*)

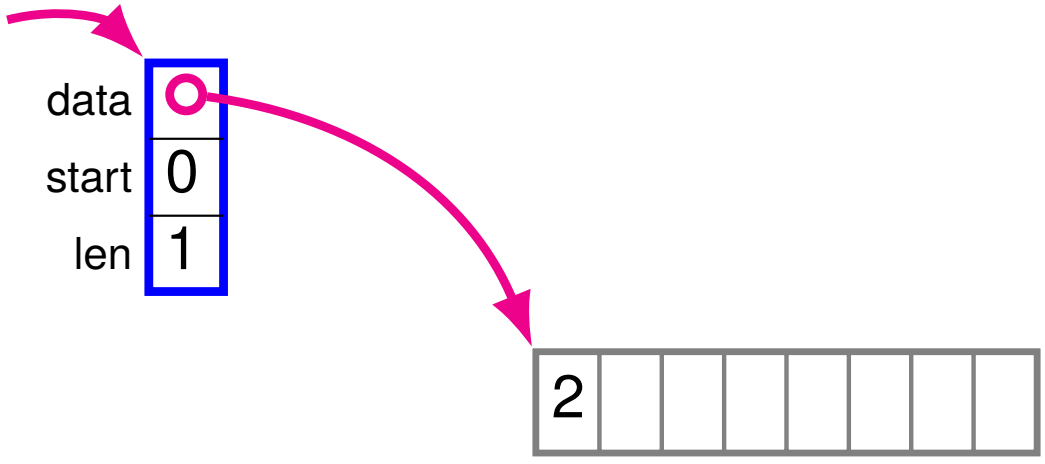


let q = RingBuffer(8)

We need a bit more infrastructure to use arrays for queues

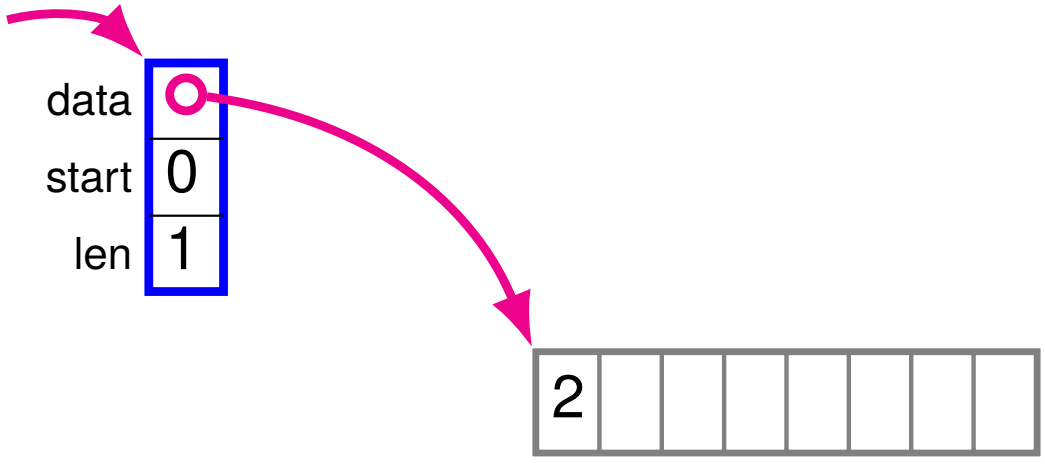
Whole new data structure: the *ring buffer*

Queue implementation: array (*ring buffer*)



let q = RingBuffer(8)
q.enqueue(2)

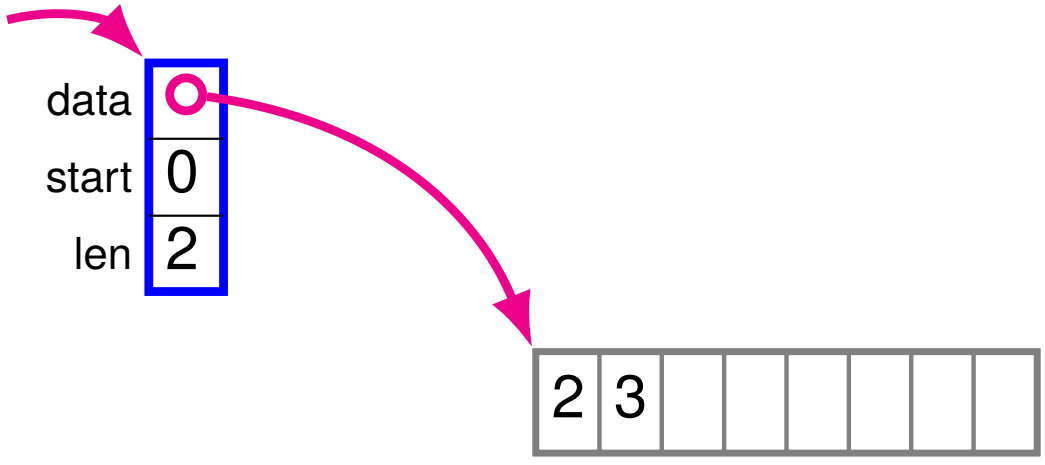
Queue implementation: array (*ring buffer*)



let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)

Add at `len`! Cheap!

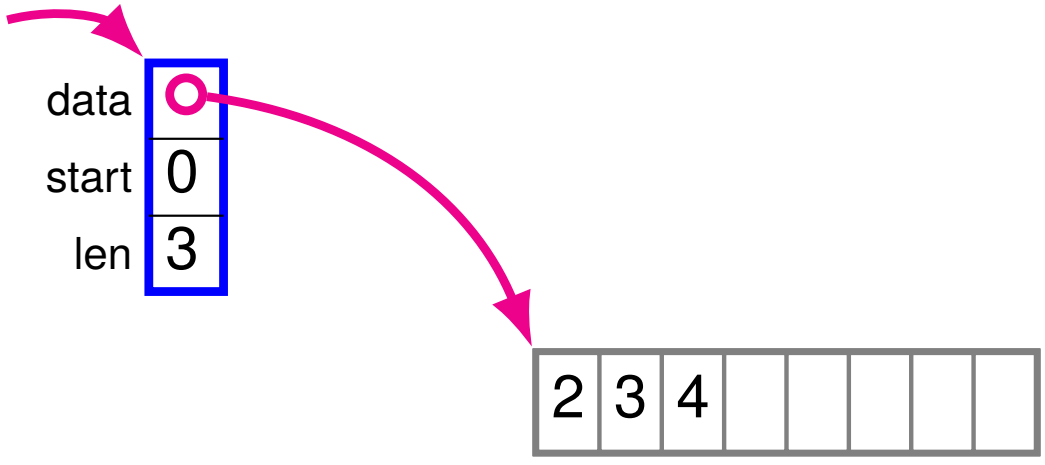
Queue implementation: array (*ring buffer*)



let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)

Add at `len`! Cheap!

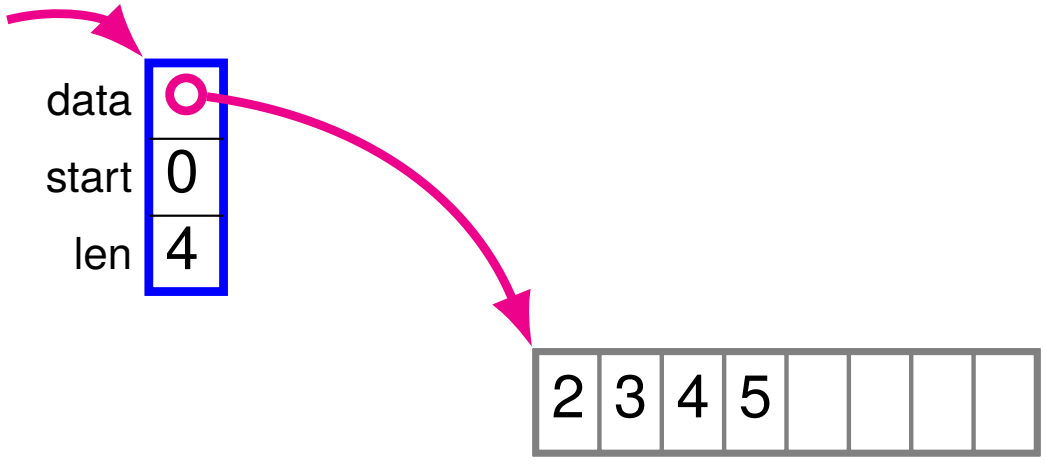
Queue implementation: array (*ring buffer*)




```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
```

Add at `len`! Cheap!

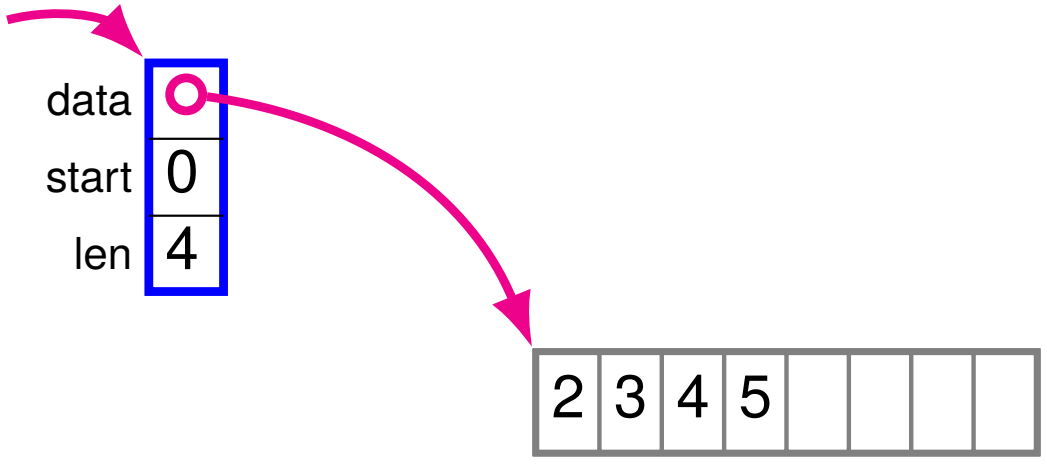
Queue implementation: array (*ring buffer*)




```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
```

Add at `len`! Cheap!

Queue implementation: array (*ring buffer*)

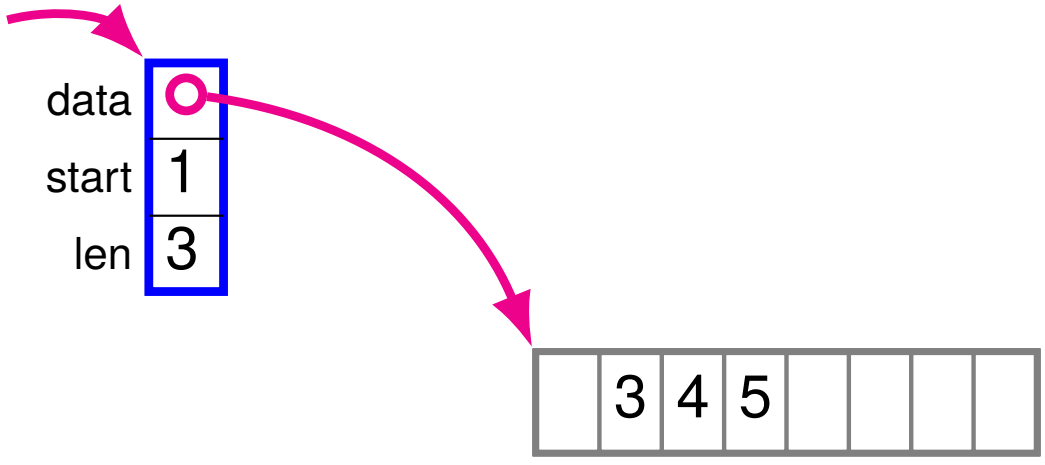



```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at `len`! Cheap!

Remove at `start`! Cheap!

Queue implementation: array (*ring buffer*)

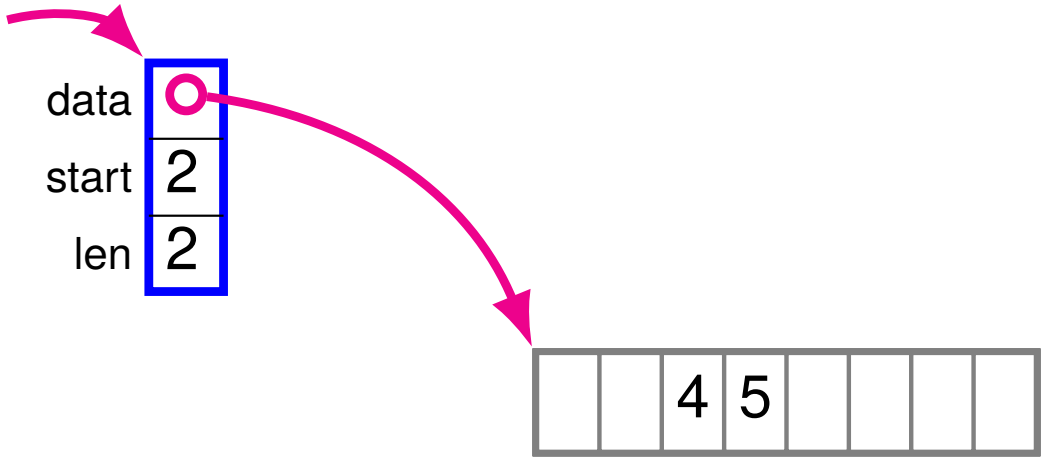



```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
```

Add at `len`! Cheap!

Remove at `start`! Cheap!

Queue implementation: array (*ring buffer*)

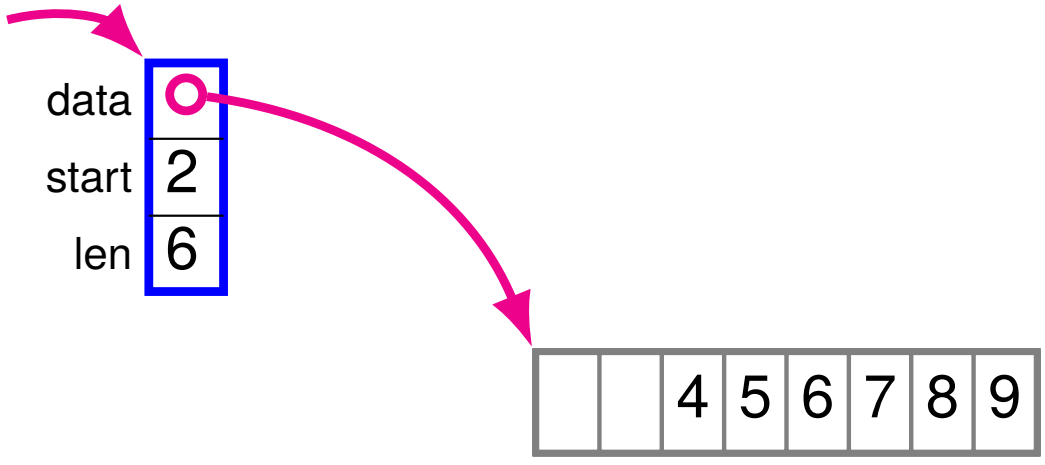



```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
q.dequeue()
```

Add at `len`! Cheap!

Remove at `start`! Cheap!

Queue implementation: array (*ring buffer*)

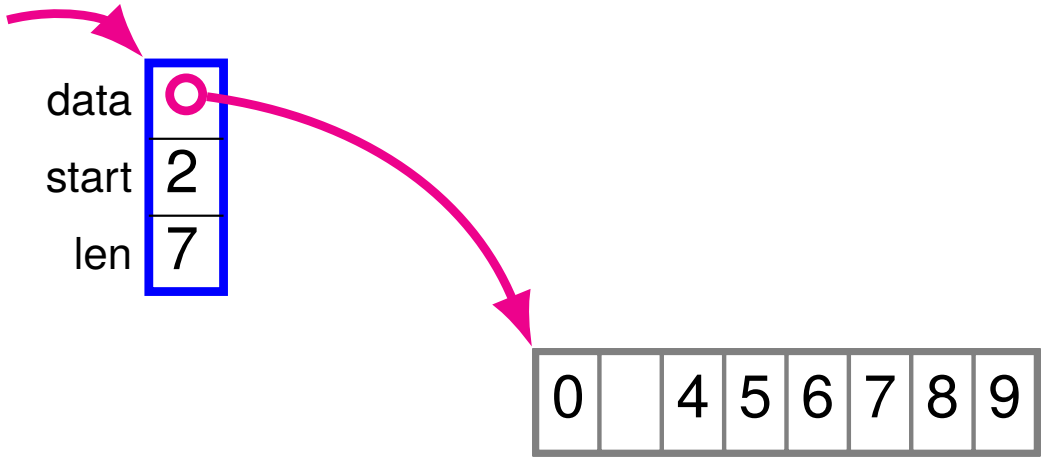



```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
q.dequeue()
⋮
```

Add at `len`! Cheap!

Remove at `start`! Cheap!

Queue implementation: array (*ring buffer*)



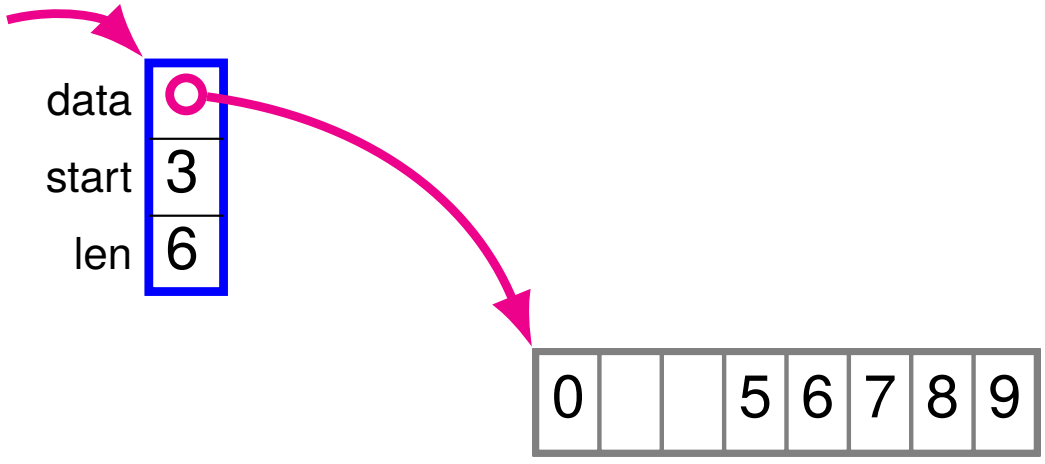

```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
q.dequeue()
⋮
q.enqueue(0)
```

Add at `len`! Cheap!

Remove at `start`! Cheap!

Add at `(start+len)%size`! Cheap!

Queue implementation: array (*ring buffer*)




```
let q = RingBuffer(8)
q.enqueue(2)
q.enqueue(3)
q.enqueue(4)
q.enqueue(5)
q.dequeue()
q.dequeue()
⋮
q.enqueue(0)
q.dequeue()
```

Add at `len`! Cheap!

Remove at `start`! Cheap!

Add at `(start+len)%size`! Cheap!

Tradeoffs: linked list queue versus ring buffer

Basically the same as for the stack implementations:

- Operations are cheap for both.
- Linked list doesn't get full.

See it in action!

David Galles (USF) designed some really nice data structure visualizations.

We'll be referring to them throughout the quarter.

You can see his stack and queue visualizations here:

- `https://www.cs.usfca.edu/~galles/visualization/StackArray.html`
- `https://www.cs.usfca.edu/~galles/visualization/StackLL.html`
- `https://www.cs.usfca.edu/~galles/visualization/QueueArray.html`
- `https://www.cs.usfca.edu/~galles/visualization/QueueLL.html`

Stop! Question time!

Before we move on to revisiting other ADTs we've seen before today...

Anything unclear so far?

Anything I missed?

ADTs we've already seen: Collections

Sets

- **Abstract values**: mathematical sets: $\boxed{\{1, 2, 3, 6\}}$, $\boxed{\{\text{"}red\text{"}, \text{"}green\text{"}, \text{"}blue\text{"}\}}$ $\boxed{\{\}}$, $\boxed{\{\{1, 2\}, \{3, 4\}\}}$, etc.
- **Abstract operations**: membership ($\in$), insertion ($+$), etc.
- **Laws** (examples):

$$\left\{s = \boxed{\{e_0, ..., e_k, ..., e_n\}}\right\} \; s.mem?(e_k) \Rightarrow \texttt{True} \; \{\}$$

$$\left\{s = \boxed{\{e_0, ..., e_n\}} \wedge e_k \notin s\right\} \; s.mem?(e_k) \Rightarrow \texttt{False} \; \{\}$$

$$\left\{s = \boxed{\{e_0, ..., e_n\}}\right\}$$

$$s.insert(e_k) \Rightarrow \texttt{None}$$

$$\left\{s = \boxed{\{e_0, ..., e_n\}} \cup \boxed{\{e_k\}}\right\}$$

- **Concrete implementations**:
  - We saw array, sorted array, linked list
  - Others are possible

Sequences

- **Abstract values**: ordered collections of items: $\boxed{[1, 5, 2, 28, -3]}$, etc.
- **Abstract operations**: get/set n^{th}*element*, add/remove at the start/end, etc.
- **Laws** (examples):

$$\left\{s = \boxed{[e_0, ..., e_k, ..., e_n]}\right\} \; s.get(k) \Rightarrow e_k \;\; \{\}$$

$$\left\{s = \boxed{[e_0, ..., e_n]}, n < k\right\} \; s.get(k) \Rightarrow error \;\; \{\}$$

$$\left\{s = \boxed{[e_0, ..., e_n]}\right\}$$

$$s.push\_front(e_k) \Rightarrow \text{None}$$

$$\left\{s = \boxed{[e_k, e_0, ..., e_n]}\right\}$$

- **Concrete implementations**:
  - Linked lists (any kind), arrays, dynamic arrays, etc.

If we have time:
Ring buffers in DSSL2

Next time: Asymptotic complexity